

Lecture 15 — Gradient Descent for Linear Regression

Given N training samples $(x^{(i)}, y^{(i)})$, linear regression seeks parameters θ^* minimising the MSE loss. The gradient points toward steepest ascent, so we step in the opposite direction.

Core Formulas

Loss function

$$L(\theta) = \frac{1}{2n} \sum_{i=1}^n \|X^{(i)}\theta - y^{(i)}\|^2$$

Gradient

$$\text{grad}_{\theta} L = \frac{1}{n} X^T (X\theta - y) = \frac{1}{n} X^T (\hat{y} - y)$$

Update rule

$$\theta \leftarrow \theta - \eta \cdot \text{grad}_{\theta} L(\theta)$$

Gradient Descent Variants — Trade-offs

Method	Samples/Step	Cost/Step	Noise	Best For
Batch GD	All n	$O(nd)$	Low	Small datasets, convex loss
SGD	1	$O(d)$	High	Online/streaming, very large n
Mini-batch	B (e.g. 32)	$O(Bd)$	Med	Deep learning (preferred)

Key insight: Mini-batch GD ($B = 32\text{--}256$) balances stable gradient estimates with cheap per-step cost — the preferred choice in deep learning. **Feature normalisation is essential** before running SGD or mini-batch GD; unnormalised large-scale features (e.g. house size in sq ft) cause enormous gradient magnitudes and wild parameter oscillations.

Numerical Example — House Price Prediction (Batch GD, Step 1)

Dataset: $n=20$ houses, $d=3$ parameters (intercept, size, bedrooms). Starting point: $\theta = (0, 0.15, 10)^T$, learning rate $\eta = 0.0001$.

Step 1 — Predict	$\hat{y}^{(i)} = 0 + 0.15 \cdot \text{size} + 10 \cdot \text{beds}$ for all 20 houses. Sum of errors = -345.6
Step 2 — Gradient	$\text{grad}_{\theta} = (-17.28, -29401.7, -89.38)$ [all negative $\Rightarrow$ θ should increase]
Step 3 — Update	θ_1 jumps 0.15 $\rightarrow$ 3.09 (large correction due to unnormalised size feature)
Real-world uses	Netflix recommendations · Energy consumption forecasting · Stock price modelling

Lecture 16 — Logistic Regression (Binary Classification)

Given a data point, output a binary label $y \in \{0, 1\}$. Logistic regression is a **discriminative model** that directly models $P(y=1 | x; \theta)$ using a hypothesis function $h_{\theta}(x)$.

Hypothesis Function — The Sigmoid

We need $h_{\theta}(x): \mathbb{R}^d \rightarrow [0,1]$ to represent a probability. Simple linear $h = \theta^T x$ fails (can exceed $[0,1]$). The sigmoid maps any real score to $(0,1)$:

$$h_{\theta}(x) = \text{sigma}(\theta^T x) = \frac{1}{1 + \exp(-\theta^T x)}$$

Property 1

$$0 \leq h_{\theta}(x) \leq 1 \text{ (valid probability)}$$

Property 2	$\theta^T x \text{ large} \Rightarrow h \rightarrow 1$ (predict $y=1$)
Property 3	$\theta^T x \text{ small} \Rightarrow h \rightarrow 0$ (predict $y=0$)
Decision rule	$\hat{y} = 1$ if $h_{\theta}(x) \geq 0.5$, i.e. $\theta^T x \geq 0$
Boundary	$\theta^T x = 0$ is the decision boundary hyperplane

Log-Odds Derivation

Odds = $h/(1-h) \in (0,\infty)$. Taking log maps to $(-\infty,+\infty)$, which we model linearly:

$$\log\left(\frac{h_{\theta}(x)}{1-h_{\theta}(x)}\right) = \theta^T x$$

Inverting this equation gives the sigmoid. Logistic regression is therefore a **linear model in log-odds space**.

Cost Function — Cross-Entropy Loss (MLE)

Derived from Maximum Likelihood Estimation under a Bernoulli model. **Convex in ω** $\Rightarrow$ gradient descent finds the global minimum.

$$J(\omega) = -(1/N) \cdot \sum [y^{(i)} \log h(x^{(i)}) + (1-y^{(i)}) \log(1-h(x^{(i)}))]$$

Gradient descent update (identical form to linear regression):

$$\omega_j \leftarrow \omega_j - (\eta/N) \cdot \sum [(h_{\omega}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}]$$

Numerical Example — University Admissions

Predict admission ($y=1$) from two exam scores. $N=5$ training samples. Initial $\omega=(0,0,0)^T$, $\eta=0.0001$.

Gradient descent iterations:

Iter	ω_0	ω_1	ω_2	$J(\omega)$
0	0.0000	0.0000	0.0000	0.6931
1	0.0000	0.0009	0.0014	0.6682
3	0.0000	0.0022	0.0035	0.6389
7	0.0000	0.0034	0.0063	0.6164
inf	-25.16	0.2060	0.2010	converged

Prediction for test point $x = (\text{Exam1}=45, \text{Exam2}=85)$:

Step 1 — Score	$z = -25.16 + 0.206 \cdot 45 + 0.201 \cdot 85 = -25.16 + 9.27 + 17.09 = 1.195$
Step 2 — Prob.	$P(y=1 x) = \text{sigma}(1.195) = 1/(1+\exp(-1.195)) \approx 0.768$
Step 3 — Classify	$0.768 \geq 0.5 \Rightarrow \hat{y} = 1$ (Admitted)

Real-World Applications

Spam filter	Word frequencies $\rightarrow P(\text{spam})$. Classify if ≥ 0.5
Medical diagnosis	Blood pressure, vitals $\rightarrow P(\text{condition present})$
Fraud detection	Transaction features $\rightarrow P(\text{fraudulent})$
Credit scoring	Financial history $\rightarrow P(\text{default})$
Ad click-through	User + ad features $\rightarrow P(\text{click})$

Quick Comparison — Linear vs Logistic Regression

Aspect	Linear Regression	Logistic Regression
Output	$\hat{y} \in \mathbb{R}$	$P(y=1) \in [0,1]$
Hypothesis	$\theta^T x$	$\sigma(\theta^T x)$
Loss	MSE (L2)	Cross-entropy
Task	Regression	Classification
Convex?	Yes	Yes
Update rule	$\theta_j \leftarrow \theta_j - \eta \frac{1}{N} \sum (h - y) x_j$	Same form as linear regression

Key Takeaways & Exam Tips

Gradient form	The update rule $\theta_j \leftarrow \theta_j - \eta \frac{1}{N} \sum (h - y) x_j$ is identical for linear and logistic regression — only the hypothesis h differs.
Decision boundary	The boundary is where $\theta^T x = 0$, NOT where $h = 0.5$ is computed.
Convexity	Cross-entropy loss is convex for logistic regression $\Rightarrow$ global minimum guaranteed.
Normalise features	Always normalise features before gradient descent. Raw large-scale features (e.g. house size in hundreds) cause huge gradient swings and slow convergence.
SGD vs Batch	SGD is unbiased on average: $E[\text{grad } L^{(i)}] = \text{grad } L$. Mini-batch $B=32-256$ is the practical sweet spot.
Log-odds insight	Logistic regression is linear in log-odds space. The sigmoid is not chosen arbitrarily — it is derived from modelling log-odds as linear.